

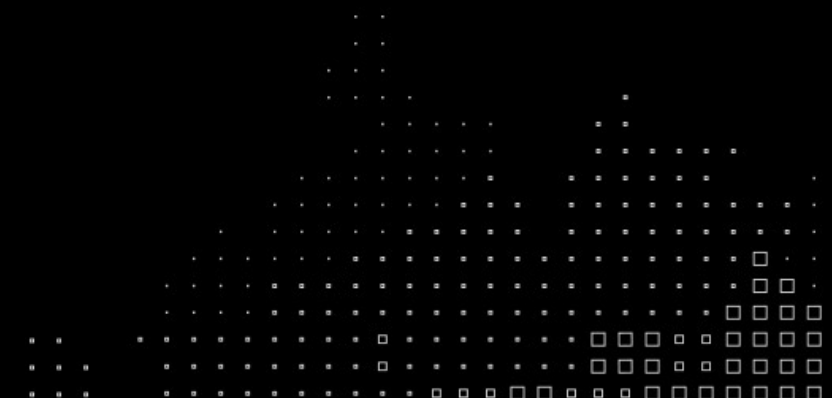
FIAP

NBA

**MBA<sup>+</sup>**

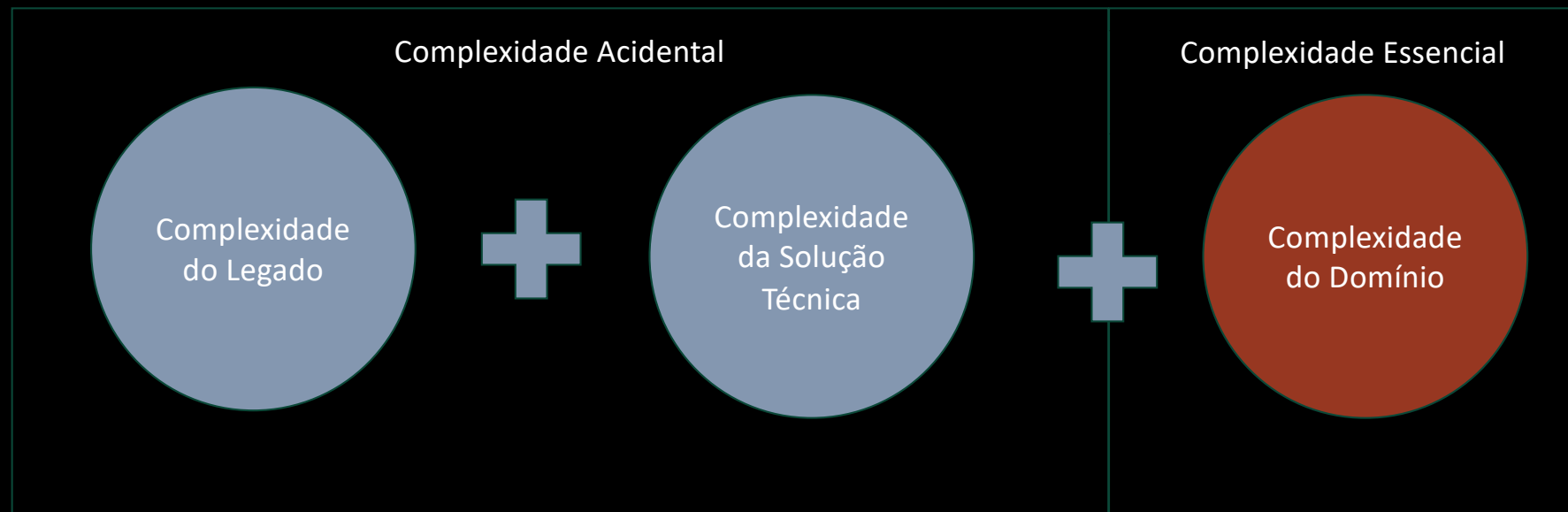
**Engenharia  
de Software**

Aula 3



# Revisão Rápida

# Complexidade de Software



*No Silver Bullet: Essence and Accidents of Software Engineering, Frederick P. Brooks, 1987*

# + Quais são os benefícios do DDD ? +

Comunicação  
Efetiva

Flexibilidade  
e  
Manutenção

Maior  
Resiliência

Modelagem  
Rica

Soluções que  
Agregam  
Valor

“Modelos de Domínio verdadeiramente bons evoluem com o tempo.”

*Eric Evans*

# Bounded Context

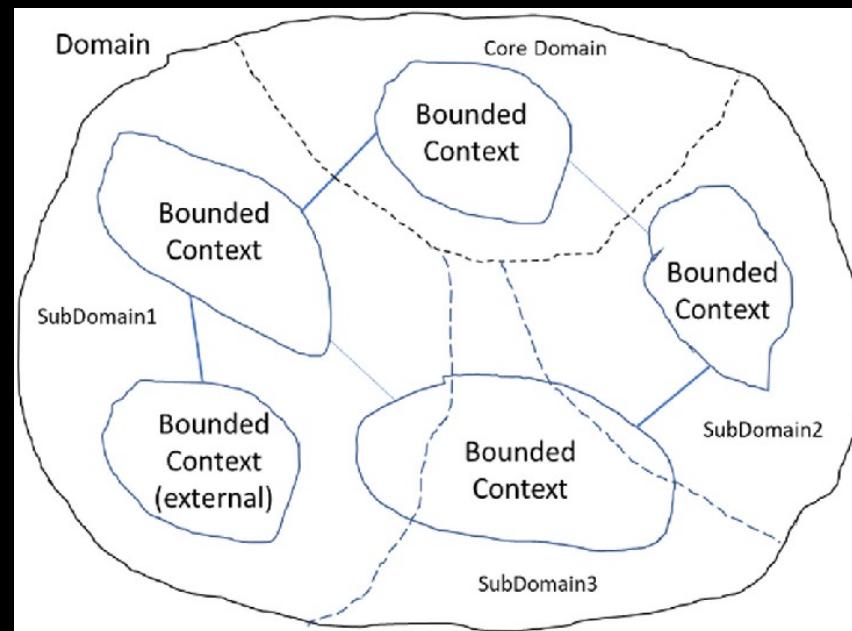
(Contexto Delimitado)

- Cada **Bounded Context** tem sua própria linguagem ubíqua.
- Ele **delimita** onde um conjunto de conceitos faz sentido.
- Ele **protege** um modelo de domínio específico, evitando ambiguidades.
- Pode se comunicar com outros contextos, mas mantém sua independência.

<https://github.com/ddd-crew/context-mapping>



# Bounded Context



<https://github.com/ddd-crew/context-mapping>



# Context Mapping

(Mapeamento de Contexto)

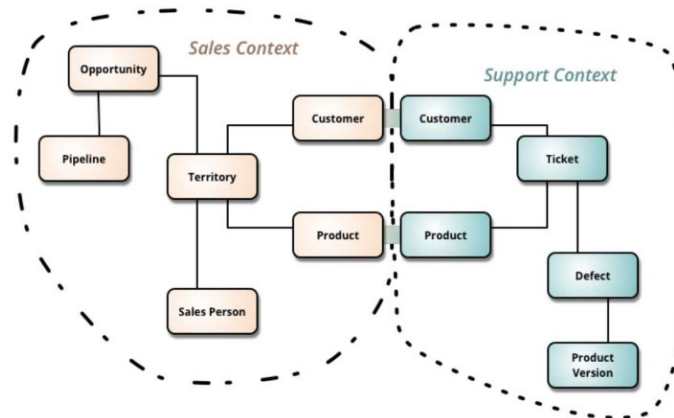
Como os Bounded Contexts interagem entre si

- Ajuda a **entender dependências** entre contextos.
- Define **estratégias de comunicação** entre times e sistemas.
- Ajuda a **evitar alto acoplamento** e **melhorar a autonomia** das equipes.



# Context Mapping

## Bounded Contexts



*DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.*

<http://martinfowler.com/bliki/BoundedContext.html>



# Tipos de Relacionamento

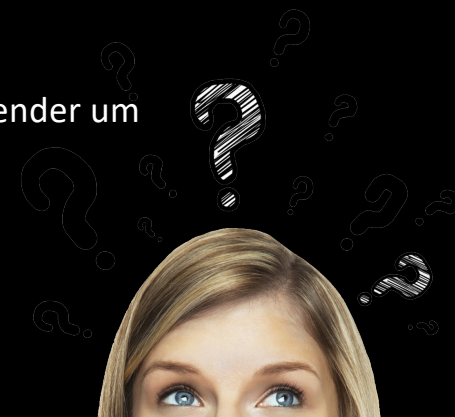
No Context Mapping, usamos **padrões** para definir como os **Bounded Contexts** interagem

## Parceria (Partnership)

- Dois contextos trabalham juntos e **dependem um do outro**.
- Mudanças em um **afetam diretamente o outro**.

**Exemplo:** Contexto de Agendamento e um Contexto de Consultas podem depender um do outro para sincronizar horários.

**Risco:** Se um time falhar, o outro pode ser prejudicado.



# Tipos de Relacionamento

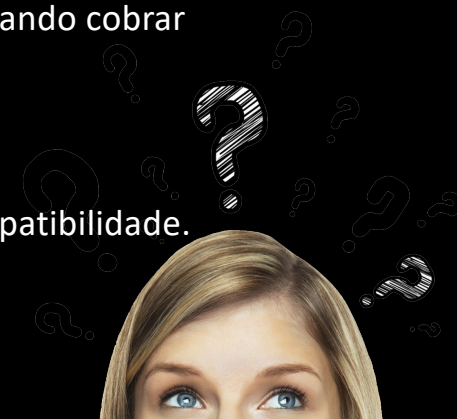
## Cliente-Fornecedor (Customer-Supplier)

Um contexto cliente depende dos dados ou serviços de um contexto fornecedor.

O fornecedor pode operar independentemente, mas mudanças nele afetam o cliente.

**Exemplo:** O Contexto de Pagamentos depende do Contexto de Consultas para saber quando cobrar uma consulta finalizada.

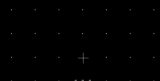
**Solução:** O cliente precisa acompanhar mudanças no fornecedor para garantir compatibilidade.



# Tipos de Relacionamento

## Kernel Compartilhado (Shared Kernel)

- Dois contextos compartilham um pequeno conjunto de regras ou dados essenciais.



**Exemplo:** O Contexto de Cadastro de Usuários pode compartilhar um modelo básico de "Usuário" com o Contexto de Autenticação.

Risco: Se o Shared Kernel crescer demais, vira um monólito disfarçado.



# Tipos de Relacionamento

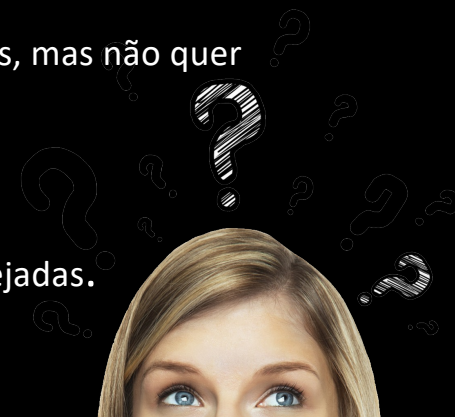
## Tradutor/Anticorruption Layer (ACL - Anti-Corruption Layer)

- Um contexto usa um adaptador para traduzir dados recebidos de outro contexto e evitar impacto direto.

```
Contexto A  
+-----+  
| ..... |  
| ..... |  
| ..... |  
| ..... |  
+-----+  
Contexto B
```

Exemplo: O Contexto de Consultas<sup>+</sup> pode precisar de dados do Contexto de Pagamentos, mas não quer depender diretamente do modelo de "Fatura".

Vantagem: Protege o modelo de domínio contra mudanças externas indesejadas.



# Tipos de Relacionamento

## Conformista (Conformist)

- Um contexto aceita o modelo do outro sem questionar.

**Exemplo:** Se o Contexto de Relatórios precisa de dados do Contexto de Consultas, ele pode apenas consumir as APIs disponíveis, sem tentar modificar nada.

Risco: Perda de controle sobre as regras de negócio, pois o conformista depende totalmente do outro contexto.



**ALGUMA PERGUNTA?**

**(POR FAVOR NÃO)**

[WWW.GERARMEMES.COM.BR](http://WWW.GERARMEMES.COM.BR)

# Aula 3

# Design Tático

Ajuda a modelar **entidades**, regras de negócio e interações dentro de um **Bounded Context**.

<https://thedomaindrivendesign.io/what-is-tactical-design/>



# Design Tático

- Entidades
- Value Objects
- Agregados
- Domínio Rico vs. Anêmico
  - Repositórios
  - Serviços de Domínio
  - Eventos de Domínio



# Design Tático

## Entidades

São objetos que possuem **identidade única** no sistema.

Mesmo que duas entidades tenham os **mesmos atributos**, se os IDs forem diferentes, são **objetos distintos**.



# Design Tático

## Value Objects

São objetos que **não possuem identidade própria**.

Dois objetos com os **mesmos valores são considerados iguais**.

São **imutáveis** e usados para representar conceitos do domínio.



# Design Tático

## Agregados

Conjunto de entidades e objetos de valor que são tratados como uma única unidade.

Definem um Aggregate Root, que controla a consistência do agregado.

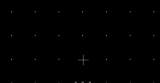
Evitam referências diretas entre entidades fora do agregado.



# Design Tático

## Domínio Rico vs. Anêmico

**Modelo Anêmico** é apenas um conjunto de propriedades, sem encapsular lógica de negócio.



**Modelo Rico** encapsula comportamento, garantindo consistência e regras de negócio dentro das entidades.



# Design Tático

## Repositórios

Responsáveis por persistir e recuperar agregados do banco de dados.

Devem trabalhar apenas com Aggregate Roots.



# Design Tático

## Serviços de Domínio

Contêm lógica de negócio que não pertence a uma entidade ou objeto de valor.

Devem ser stateless (sem estado interno).



# Design Tático

## Eventos de Domínio

Notificam mudanças no domínio para outros componentes.

Evitam acoplamento entre objetos.



# Design Tático

## Exemplos

[Link](#)



The Kahoot! logo is centered on a black background. The logo itself is a square divided into four quadrants by a vertical and a horizontal line. The top-left quadrant is red, the top-right is blue, the bottom-left is yellow, and the bottom-right is green. Overlaid on this square is a faint, light-colored world map. The word "Kahoot!" is written in a large, white, bold, sans-serif font across the center of the square.

# Kahoot!

# Dinâmica:

DDD-3



FIAP

NBA

MBA<sup>+</sup>

Copyright © 2024 Prof. Thiago Keller

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).

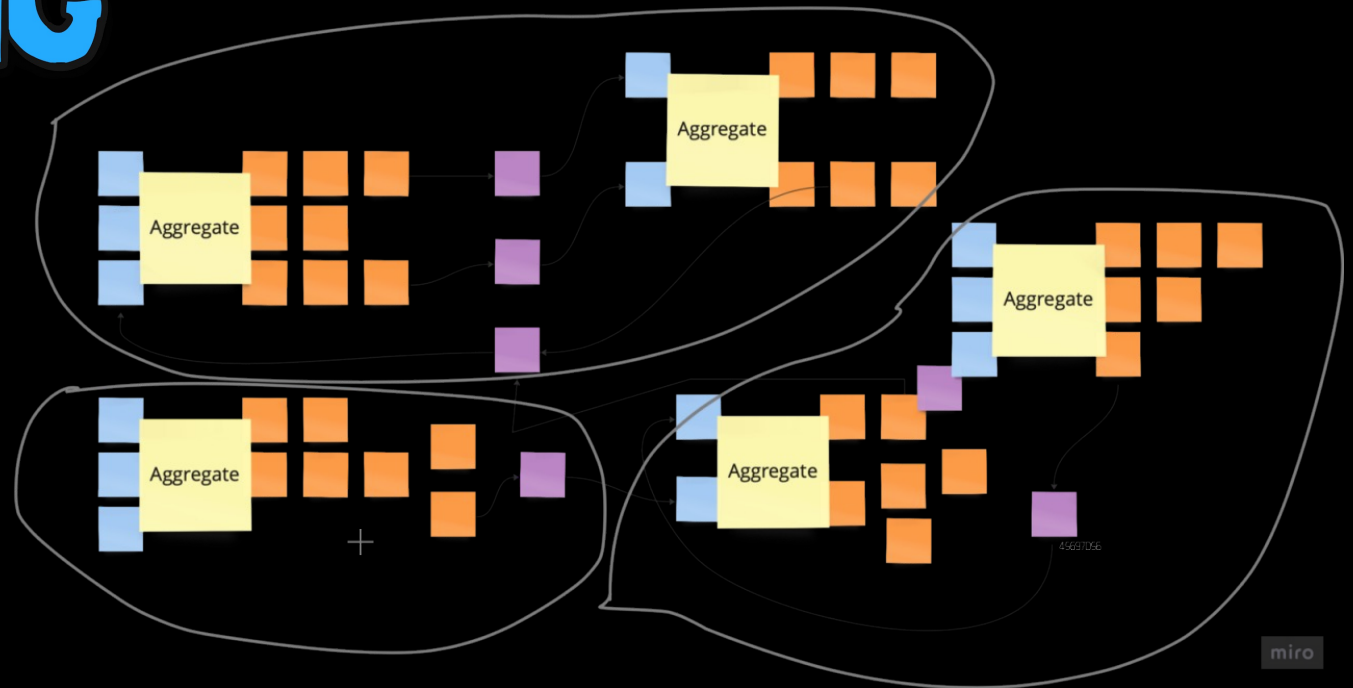
# + Dinâmica: EventStorming +



- Reúnam-se em grupos
- **Mapeamento do Problema (CORE DOMAIN)**
- Mapeamento dos Termos e Conceitos
- Organização e Agrupamento de Conceitos
- Criação de Relações com Setas e Conexões
- Refinamento da Linguagem Onipresente

# EVENT STORMiNG

<https://www.eventstorming.com/>



<https://www.linkedin.com/pulse/o-que-%C3%A9-eventstorming-e-como-este-formato-de-workshop-rodrigues/?originalSubdomain=pt>

